1. Program to calculate the area of triangle.

```
"""area = 1/2*b*h
b: base
h: height"""
b=float(input("enter base: "))
h=float(input("enter height: "))
a = (1/2)*b*h
print("area of triangle is ",a)
```

OUTPUT:

```
enter base: 4
enter height: 5
area of triangle is  10.0

Process finished with exit code 0
```

2. Program to calculate the Compound Interest.

```
"""
amount = p(1+r/100)**t
p: principal
r: rate
t: time
compound interest = amount - principal"""

principal = float(input("enter principal: "))
rate = float(input("enter rate: "))
time = float(input("enter time: "))
#amount = principal*(1+rate/100) ** time
amount2 = principal*pow((1+rate/100), time)
print(round(amount2,2))
ci = amount2 - principal
print("compound interest",round(ci,2))
```

OUTPUT:

```
enter principal: 1000
enter rate: 10
enter time: 4
1464.1
compound interest 464.1

Process finished with exit code 0
```

3. Program to calculate Simple Interest.

```
"""
Simple Interest = p*r*t/100
p: principal amount
r: rate of interest
t: time duration
"""

principal = float(input("enter principal amount: "))
rate = float(input("enter rate of interest: "))
time = float(input("enter time duration: "))
si = (principal*rate*time)/100
print("simple interest: ",si)
```

OUTPUT:

```
enter principal amount: 1000
enter rate of interest: 4
enter time duration: 3
simple interest:  120.0

Process finished with exit code 0
```

4. Program to calculate the area and perimeter of Triangle using sides.

```
"""
when all the side of tringle is known - a,b,c
semi perimeter (s) = (a+b+c)/2
area = square root of (s*(s-a)*(s-b)*(s-c))
"""
a=float(input("enter 1st side of tringle: "))
b=float(input("enter 2nt side of tringle: "))
c=float(input("enter 3rd side of tringle: "))
s= (a+b+c)/2
area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
print(f"The area of the tringle is ",round(area,2))
print("semi perimeter of the tringle is ",round(s,2))
```

OUTPUT:

```
enter 1st side of tringle: 2
enter 2nt side of tringle: 3
enter 3rd side of tringle: 4
The area of the tringle is  2.9
semi perimeter of the tringle is  4.5

Process finished with exit code 0
```

5. Program to Add, Multiply, Subtract and Division of two number.

```
num1 = int(input("enter first number: "))
num2 = int(input("enter second number: "))
addition = num1 + num2
subtraction = num1 - num2
multiplication = num1 * num2
division = num1 / num2
print("Addition: ",addition,"\nSubtraction: ",subtraction,"\nMultiplication: ",multiplication,"\nDivision: ",division)
```

OUTPUT:

```
enter first number: 2
enter second number: 4
Addition:  6
Subtraction:  -2
Multiplication:  8
Division:  0.5

Process finished with exit code 0
```

6. Program to concatenate two string.

```
first_name=input("enter your first name: ")
last_name=input("enter your last name: ")
full_name=first_name+" "+last_name+"!"
print("hello,",full_name,"Welcome to the python program.")
```

OUTPUT:

```
enter your first name: abdul
enter your last name: majeed
hello, abdul majeed! Welcome to the python program.

Process finished with exit code 0
```

7. Different operations on Lists using different methods.

```
#  slicing of list
# syntax: list[start index,end index,steps]

l1 = [1,6,5,7,89,3,4,2,67.9]
print(l1[1:6:1])

# concatination of lists/joining of lists
# operator: +
l2 = [1,4,7]
l3 = [2,8]
print(l2+l3)

# repetation of list
# operators: *
print(l2 * 3)

# append()
# this function add item at the end of list
# syntax: list.append(item)

fruits = ["apple", "banana", "cherry"]
print(fruits)
# print(fruits.append("orange"))#it give us none
# print(fruits.pop())
# print(fruits.pop())
fruits.append("orange")
print(fruits)

# remove()
# syntax: list.remove(item)
fruits.remove("orange")
print(fruits)


# insert()
# add an element before the specified index
# syntax: list.insert(index,item)

fruits.insert(1,"orange")
print(fruits)
```

OUTPUT:

```
[6, 5, 7, 89, 3]
[1, 4, 7, 2, 8]
[1, 4, 7, 1, 4, 7, 1, 4, 7]
['apple', 'banana', 'cherry']
['apple', 'banana', 'cherry', 'orange']
['apple', 'banana', 'cherry']
['apple', 'orange', 'banana', 'cherry']

Process finished with exit code 0
```

8. Basics operations on list.

```
name = ("Abdul")
age = 20
percent = 80.5

#list in python
student =["Abdul", 20, 80.5,]
print(type(student))
print(student)

days_of_week = ["mon", "tue", "wed", "thur", "fri", "sat", "sun"]
print(days_of_week[0])
print(days_of_week[4])
print(days_of_week[6])
print("the last day of weeks is:", days_of_week[-1])

# length of the list => the numbers of items/element in the list
print(len(days_of_week))
```

OUTPUT:

```
<class 'list'>
['Abdul', 20, 80.5]
mon
fri
sun
the last day of weeks is: sun
7

Process finished with exit code 0
```

9. Operations on nested list.

```
# list inside the list

l1 = [5, 1.5, "pyrhon", True, None, [1,2,3],10]
# print(len(l1))

print(l1[-2][0])

l2 = [[1,2],[5,6],[3,4,[0,7]]]
print(len(l2))
print(l2[-1][-1][-1])# fetching 7 from this
```

OUTPUT:

```
1
3
7

Process finished with exit code 0
```

10. Operations on string using fstring.

```
name = "abdul"
age = 20
language = "python"
hours = 3
# abdul is 20 years old. He studies python 3 hours a day,
print(name," is ",age, "years old. he studies ",language ,hours," hours a day")

# using f_string
print(f"{name} is {age} years old. He studies {language} {hours} hours a day ")

sub1 = 23
sub2 = 56
sub3 = 89

print(f"{name} scored {sub1+sub2+sub3} marks in total")
percent = (sub1+sub2+sub3)/3
print(f"{name} got {percent}%")
```

OUTPUT:

```
abdul  is  20 years old. he studies  python 3  hours a day
abdul is 20 years old. He studies python 3 hours a day
abdul scored 168 marks in total
abdul got 56.0%

Process finished with exit code 0
```

11. Basics operation on string.

```
# counting substring from string
# count()
# syntax: string.count(substring)

s1 = "we are learning python. python is fun"
s2 = "e"
print(f"occurences of {s2} is {s1.count(s2)}")
s3 = "python"
print(f"occurences of {s3} is {s1.count(s3)}")
s4 = " "
print(f"occurences of spaces is {s1.count(s4)}")


#Changing case of string from upper case to lower case and visa versa
#upper(), lower(), title(), capitalize()

s1 = "Python3.14.3"
s2 = "PYTHON"
print(s1.upper())
print(s1.lower())
print(s2.capitalize())
s3 = "we are learning python"
print(s3.title())


# Starting and ending of a string
s1 = "we are learning python"

# startswith()
# syntax: string.stratswith(substring)

print(s1.startswith("w"))# it give us true
print(s1.startswith("we are"))# it give us true
print(s1.startswith("are"))# it give us false
print(s1.startswith("We are"))# it give us false because python is case sansitive language

# endswith
# syntax: string.endswith(substring)

print(s1.endswith("n"))# it give us true
print(s1.endswith("python"))# it give us true
print(s1.endswith("ing"))# it give us false
print(s1.endswith("N"))# it give us false because python is case sansitive language
```

OUTPUT:

```
occurences of e is 3
occurences of python is 2
occurences of spaces is 6
PYTHON3.14.3
python3.14.3
Python
We Are Learning Python
True
True
False
False
True
True
False
False

Process finished with exit code 0
```

12. Write a program to implement Linear Search on an array.

```
def linear_search(arr, target):
  for i in range(len(arr)):
    if arr[i] == target:
      return i
  return -1

# Usage
data = [10, 23, 45, 70, 11, 15]
print(f"Linear Search: Element found at index {linear_search(data, 11)}")
```

OUTPUT:

```
"C:\Users\HP VICTUS\AppData\Local\Python\
Linear Search: Element found at index 4

Process finished with exit code 0
```

13. Write a program to implement Binary Search using iteration.

```
def binary_search_iterative(arr, target):
    """
    Performs an iterative binary search on a sorted list.
    Returns the index of the target if found, otherwise returns -1.
    """
    low = 0
    high = len(arr) - 1

    while low <= high:
        # Calculate the middle index
        mid = (low + high) // 2

        # Check if target is present at mid
        if arr[mid] == target:
            return mid

        # If target is greater, ignore left half
        elif arr[mid] < target:
            low = mid + 1

        # If target is smaller, ignore right half
        else:
            high = mid - 1

    # If we reach here, the element was not present
    return -1


# --- Test the Program ---
if __name__ == "__main__":
    sample_list = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
    search_val = 23

    result = binary_search_iterative(sample_list, search_val)

    if result != -1:
        print(f"Element {search_val} found at index: {result}")
    else:
        print(f"Element {search_val} is not present in the array.")
```

OUTPUT:

```
"C:\Users\HP VICTUS\AppData\Local\Python
Element 23 found at index: 5

Process finished with exit code 0
```

14. Write a program to implement Binary Search using recursion.

```
def binary_search_recursive(arr, low, high, target):
    """
    Performs a recursive binary search on a sorted list.
    """
    # Base Case: Search space is exhausted
    if low > high:
        return -1
    # Calculate middle index
    mid = (low + high) // 2
    # Case 1: Target found at mid
    if arr[mid] == target:
        return mid
    # Case 2: Target is in the left half
    elif arr[mid] > target:
        return binary_search_recursive(arr, low, mid - 1, target)
    # Case 3: Target is in the right half
    else:
        return binary_search_recursive(arr, mid + 1, high, target)
# --- Test the Program ---
if __name__ == "__main__":
    data = [3, 9, 14, 19, 25, 33, 48, 62]
    val = 62

    # Initial call with low=0 and high=last index
    result = binary_search_recursive(data, 0, len(data) - 1, val)

    if result != -1:
        print(f"Element {val} found at index: {result}")
    else:
        print(f"Element {val} not found.")
```

OUTPUT:

```
"C:\Users\HP VICTUS\AppData\Local\F
Element 62 found at index: 7

Process finished with exit code 0
```

15. Write a program to sort an array using Bubble Sort.

```
def bubble_sort(arr):
    """
    Sorts an array using the Bubble Sort algorithm with an optimization
    to stop early if the list becomes sorted.
    """
    n = len(arr)
    for i in range(n):
        # Flag to check if any swapping happened in this pass
        swapped = False
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Compare adjacent elements
            if arr[j] > arr[j + 1]:
                # Swap if they are in the wrong order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # If no two elements were swapped by inner loop, then break
        if not swapped:
            break
    return arr
# --- Test the Program ---
if __name__ == "__main__":
    data = [64, 34, 25, 12, 22, 11, 90]
    print("Original array:", data)

    sorted_data = bubble_sort(data)
    print("Sorted array:  ", sorted_data)
```

OUTPUT:

```
"C:\Users\HP VICTUS\AppData\Local\Python\python
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array:   [11, 12, 22, 25, 34, 64, 90]

Process finished with exit code 0
```

16. Write a program to sort an array using Selection Sort.

```
def selection_sort(arr):
    """
    Sorts an array using the Selection Sort algorithm.
    """
    n = len(arr)

    # Traverse through all array elements
    for i in range(n):
        # Find the minimum element in remaining unsorted array
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap the found minimum element with the first element of unsorted part
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr

# --- Test the Program ---
if __name__ == "__main__":
    data = [29, 10, 14, 37, 13]
    print("Original array:", data)

    sorted_data = selection_sort(data)
    print("Sorted array:  ", sorted_data)
```

OUTPUT:

```
"C:\Users\HP VICTUS\AppData\Local\Python\
Original array: [29, 10, 14, 37, 13]
Sorted array:   [10, 13, 14, 29, 37]

Process finished with exit code 0
```

17. Write a program to sort an array using Insertion Sort.

```
def insertion_sort(arr):
    """
    Sorts an array using the Insertion Sort algorithm.
    """
    # Start from the second element (index 1)
    for i in range(1, len(arr)):
        key = arr[i]

        # Move elements of arr[0..i-1] that are greater than key
        # to one position ahead of their current position
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1

        # Place the key at its correct position
        arr[j + 1] = key

    return arr


# --- Test the Program ---
if __name__ == "__main__":
    data = [12, 11, 13, 5, 6]
    print("Original array:", data)

    sorted_data = insertion_sort(data)
    print("Sorted array:  ", sorted_data)
```

OUTPUT:

```
"C:\Users\HP VICTUS\AppData\Local\Python
Original array: [12, 11, 13, 5, 6]
Sorted array:   [5, 6, 11, 12, 13]

Process finished with exit code 0
```

18. Write a program to sort an array using Merge Sort.

```
def merge_sort(arr):
  """
  Sorts an array using the Merge Sort algorithm (Recursive).
  """
  if len(arr) > 1:
    # Finding the mid of the array
    mid = len(arr) // 2

    # Dividing the array elements into 2 halves
    L = arr[:mid]
    R = arr[mid:]

    # Sorting the first half
    merge_sort(L)

    # Sorting the second half
    merge_sort(R)

    i = j = k = 0

    # Copy data to temp arrays L[] and R[]
    while i < len(L) and j < len(R):
      if L[i] < R[j]:
        arr[k] = L[i]
        i += 1
      else:
        arr[k] = R[j]
        j += 1
      k += 1

    # Checking if any element was left in L
    while i < len(L):
      arr[k] = L[i]
      i += 1
      k += 1

    # Checking if any element was left in R
    while j < len(R):
      arr[k] = R[j]
      j += 1
      k += 1

  return arr
```

```
# --- Test the Program ---
if __name__ == "__main__":
    data = [38, 27, 43, 3, 9, 82, 10]
    print("Original array:", data)

    sorted_data = merge_sort(data)
    print("Sorted array:  ", sorted_data)
```

OUTPUT:

```
"C:\Users\HP VICTUS\AppData\Local\Python\python
Original array: [38, 27, 43, 3, 9, 82, 10]
Sorted array:   [3, 9, 10, 27, 38, 43, 82]

Process finished with exit code 0
```

19. Write a program to sort an array using Quick Sort.

```
def quick_sort(arr):
    """
    Sorts an array using the Quick Sort algorithm.
    Uses list comprehensions for clarity.
    """
    # Base case: if the array has 0 or 1 element, it is already sorted
    if len(arr) <= 1:
        return arr

    # Selecting the middle element as the pivot
    pivot = arr[len(arr) // 2]

    # Elements smaller than the pivot
    left = [x for x in arr if x < pivot]
    # Elements equal to the pivot
    middle = [x for x in arr if x == pivot]

    # Elements greater than the pivot
    right = [x for x in arr if x > pivot]

    # Recursively sort left and right, then combine
    return quick_sort(left) + middle + quick_sort(right)
# --- Test the Program ---
if __name__ == "__main__":
    data = [10, 80, 30, 90, 40, 50, 70]
    print("Original array:", data)
    sorted_data = quick_sort(data)
    print("Sorted array:  ", sorted_data)
```

OUTPUT:

```
"C:\Users\HP VICTUS\AppData\Local\Python\python
Original array: [10, 80, 30, 90, 40, 50, 70]
Sorted array:   [10, 30, 40, 50, 70, 80, 90]

Process finished with exit code 0
```

20. Program to create a dictionary.

```
# creating a dictionary
country_capitals = {
  "Germany": "Berlin",
  "Canada": "Ottawa",
  "England": "London"
}
# printing the dictionary
print(country_capitals)
```

OUTPUT:

```
{'Germany': 'Berlin', 'Canada': 'Ottawa', 'England': 'London'}

Process finished with exit code 0
```

21. Program to change dictionary items.

```
country_capitals = {
  "Germany": "Berlin",
  "Italy": "Naples",
  "England": "London"
}

# change the value of "Italy" key to "Rome"
country_capitals["Italy"] = "Rome"

print(country_capitals)
```

OUTPUT:

```
{'Germany': 'Berlin', 'Italy': 'Rome', 'England': 'London'}

Process finished with exit code 0
```

22. Program to add items in dictionary.

```
country_capitals = {
 "Germany": "Berlin",
 "Canada": "Ottawa",
}

# add an item with "Italy" as key and "Rome" as its value
country_capitals["Italy"] = "Rome"

print(country_capitals)
```

OUTPUT:

```
{'Germany': 'Berlin', 'Canada': 'Ottawa', 'Italy': 'Rome'}

Process finished with exit code 0
```

23. Program to access dictionary items.

```
country_capitals = {
 "Germany": "Berlin",
 "Canada": "Ottawa",
 "England": "London"
}
# access the value of keys
print(country_capitals["Germany"])   # Output: Berlin
print(country_capitals["England"])   # Output: London
```

OUTPUT:

```
Berlin
London

Process finished with exit code 0
```

24. Program to find length of dictionary.

```
country_capitals = {"England": "London", "Italy": "Rome"}

# get dictionary's length
print(len(country_capitals))   # Output: 2

numbers = {10: "ten", 20: "twenty", 30: "thirty"}

# get dictionary's length
print(len(numbers))          # Output: 3

countries = {}

# get dictionary's length
print(len(countries))        # Output: 0
```

OUTPUT:

```
2
3
0

Process finished with exit code 0
```

25. Program to remove all items from dictionary at once.

```
country_capitals = {
  "Germany": "Berlin",
  "Canada": "Ottawa",
}

# clear the dictionary
country_capitals.clear()

print(country_capitals)
```

OUTPUT:

```
{}

Process finished with exit code 0
```

26. Program to remove item from dictionary.

```
country_capitals = {
  "Germany": "Berlin",
  "Canada": "Ottawa",
}

# delete item having "Germany" key
del country_capitals["Germany"]

print(country_capitals)
```

OUTPUT:

```
{'Canada': 'Ottawa'}

Process finished with exit code 0
```

27. Operations on dictionary using membership operators.

```
file_types = {
   ".txt": "Text File",
   ".pdf": "PDF Document",
   ".jpg": "JPEG Image",
}

# use of in and not in operators
print(".pdf" in file_types)      # Output: True
print(".mp3" in file_types)      # Output: False
print(".mp3" not in file_types)   # Output: True
```

OUTPUT:

```
True
False
True

Process finished with exit code 0
```

28. Program to create nested dictionary.

```
people = {1: {'name': 'John', 'age': '27', 'sex': 'Male'},
        2: {'name': 'Marie', 'age': '22', 'sex': 'Female'}}

print(people)
```

OUTPUT:

```
{1: {'name': 'John', 'age': '27', 'sex': 'Male'}, 2: {'name': 'Marie', 'age': '22', 'sex': 'Female'}}

Process finished with exit code 0
```

29. Program to add another dictionary in nested dictionary.

```
people = {1: {'name': 'John', 'age': '27', 'sex': 'Male'},
    2: {'name': 'Marie', 'age': '22', 'sex': 'Female'},
    3: {'name': 'Luna', 'age': '24', 'sex': 'Female', 'married': 'No'}}

people[4] = {'name': 'Peter', 'age': '29', 'sex': 'Male', 'married': 'Yes'}
print(people[4])
```

OUTPUT:

```
{'name': 'Peter', 'age': '29', 'sex': 'Male', 'married': 'Yes'}

Process finished with exit code 0
```

30. Program to access element from nested dictionary.

```
people = {1: {'name': 'John', 'age': '27', 'sex': 'Male'},
    2: {'name': 'Marie', 'age': '22', 'sex': 'Female'}}

print(people[1]['name'])
print(people[1]['age'])
print(people[1]['sex'])
```

OUTPUT:

```
John
27
Male

Process finished with exit code 0
```